

RESEARCH ARTICLE

TDMiL: Tiny Distributed Machine Learning for Microcontroller-Based Interconnected Devices

MAYANK GULATI¹, KOEN ZANDBERG¹, ZHAOLAN HUANG¹,
GERHARD WUNDER¹, (Senior Member, IEEE), CEDRIC ADJIH²,
AND EMMANUEL BACCELLI^{1,2}

¹Institute of Computer Science, Freie Universität Berlin, 14195 Berlin, Germany

²Inria, 91120 Palaiseau, France

Corresponding author: Mayank Gulati (mayank.gulati@fu-berlin.de)

This work was supported in part by the MESRI-BMBF German/French Cybersecurity Program under Grant ANR-20-CYAL-0005 and Grant 16KIS1395K. The work of Emmanuel Baccelli was supported by the Einstein Center Digital Future (ECDF) Berlin, from Qualcomm France, and from France 2030.

ABSTRACT More and more, edge devices embark Artificial Neuron Networks. In this context, a trend is to simultaneously decentralize their training as much as possible while shrinking their resource requirements, both for inference and training—tasks that are typically intensive in terms of data, memory, and computation. At the edge's extremity, a specific challenge arises concerning the inclusion of microcontroller-based devices typically deployed in the IoT. So far, no general framework has been provided for that. Such devices not only have extremely challenging resource constraints (weak CPUs, slow network connections, memory budgets measured in kilobytes) but also exhibit high polymorphism, leading to large variability in computational performance among these devices. In this paper, we design and implement TDMiL, a versatile framework for distributed training, and transfer learning. TDMiL interconnects and combines logical components including CoAPerator (a central aggregator) and various tiny embedded software runtimes that are specifically tailored for networks comprising heterogeneous, resource-constrained devices built on diverse types of microcontrollers. We report on experiments conducted with the TDMiL framework, which we use to comparatively evaluate several schemes devised to address computational variability among distributed learning microcontroller-based devices, i.e., stragglers. Additionally, we release the code of our implementation of TDMiL as an open-source project, which is compatible with common commercial off-the-shelf IoT hardware and a well-known open-access IoT testbed.

INDEX TERMS Distributed learning, federated learning (FL), Internet of Things (IoT), machine learning, microcontrollers, TinyML-as-a-Service (TMLaaS).

I. INTRODUCTION

Machine learning (ML), particularly when leveraging deep neural networks (DNNs), has undergone a significant transformation over the last decade, making an indelible impact across various sectors. The standard deployment of ML involves a data pipeline that utilizes a model—a sophisticated framework of algorithms that process and interpret data to make informed decisions. Such models are initially subjected to an intensive training phase, equipping

them for subsequent inference operations and their eventual production use. Initially requiring powerful PCs and clusters, machine learning has been adapted to less powerful edge devices such as smartphones, or single board computers in the category of Jetson Nano or RaspberryPi, which are based on relatively cheap microprocessors with a power consumption in the range of a few Watts. Some prior work on distributed learning on such machines can be found for instance in [1] and [2]. However, pushing further, the TinyML [3] community is now targeting even smaller devices based on *microcontrollers*. Despite their very limited computing capacity and a much lower power consumption

The associate editor coordinating the review of this manuscript and approving it for publication was Nitin Gupta¹.

in the range of milliWatts, microcontrollers can perform model inference effectively once they have been adequately trained and optimized off-device (this training phase usually demands considerable data and more computing resources).

The pioneering efforts of the TinyML community have spearheaded the investigation into on-device learning such as MCUNet [4], etc., especially focusing on extremely resource-constrained microcontrollers. As discussed in [5], addressing memory and power constraints in microcontroller units for Internet of Things (IoT) has been a driving force behind TinyML applications. The TinyML-as-a-Service (TMLaaS) architecture supports on-device learning, as demonstrated through a practical case study. Additionally, [6] explores the potential of reformable TinyML, emphasizing adaptability and cognition in IoT devices. The survey paper categorizes solutions, outlining their autonomous self-enhancement capabilities and highlighting the roles of distributed TinyML and over-the-air approaches.

Altogether, these works underscore the utility of TinyML in providing lightweight ML solutions for resource-constrained IoT environments. Among these advancements, Federated Learning (FL) emerges as a decentralized ML technique that trains models across numerous collaborating devices without centralized training, thereby safeguarding privacy and enabling local customization. By leveraging such decentralized strategies, the ML landscape is witnessing a transformative shift, enabling global model training without any exposing, transmitting (centralizing) of raw data. This influential approach initially introduced in a seminal paper on Federated Averaging [7], with subsequent research such as [8] focusing on communication efficiency and enhanced privacy via strategies such as sub-sampling and quantization.

The typical FL workflow involves selecting participating devices, distributing a global model, training locally, sending updates to a central server, aggregating for a new global model, and iterating until convergence. This collaborative learning process ensures that the resulting model is refined and suitable for inference across all participating devices. Throughout the FL process, communication protocols play a pivotal role.

In addition to the general challenges encountered in FL, such as heterogeneity stemming from differences in client systems, including hardware resources and network availability (*system heterogeneity*), as well as variations in data distributions across these devices (*statistical heterogeneity*), tiny distributed ML must confront the overarching problem of resource constraints. Moreover, solutions like FedProx [9], aimed at mitigating heterogeneity under less constrained conditions, raise questions. They necessitate the tracking of both global and local models during a client's update, posing challenges in terms of its direct applicability.

A. CHALLENGES IN DISTRIBUTED LEARNING WITH INTERCONNECTED MICROCONTROLLERS

Current research into FL for edge devices often glosses over a critical aspect—the in-depth analysis of computational

and communication costs—imperative for devices with limited resources. Additionally, EdgeML's [10] work on multi-hop wireless network-based learning underscores the need for a robust communication stack for collaborative learning, emphasizing that much of the existing research primarily focuses on optimizing computational performance, neglecting the importance of the communication layer.

More loosely related work on Gossip Learning (GL) aims to further decentralize model aggregation, in a peer-to-peer fashion. However, studies such as [11] point towards the limitations of the GL framework, notably in terms of delayed convergence, especially when dealing with diverse data distributions. Further work on GossipFL [12] focused on improving the convergence rate and accuracy in constrained IoT environments.

Considering the diverse nature of wireless networks, differences in hardware specifications among edge devices, and varying data distributions across these devices, we can make two key observations:

- The presence of computational variability entails the occurrence of slow-performing workers, commonly termed “*stragglers*”. If not dealt with effectively, these stragglers have the potential to significantly impact the overall convergence of the training process, as indicated in FedProx [9].
- Communication protocols need to be optimized not only for resource constraints but also to be adaptable across a diverse range of link layers. This includes typical wireless technologies like Bluetooth Low Energy (BLE), IEEE 802.15.4, and LoRa, as well as various network technologies, including wired options such as BACnet, PLC as specified in [13], and local bus communications (USB, Inter-Integrated Circuit (I²C), etc.). Ensuring the use of secure protocols is crucial, especially when handling the exchange of (privacy) sensitive information among tiny devices reliably.

It's important to note that existing studies in the domain of low-resource distributed machine learning have not been specifically designed to tackle these particular challenges. Additionally, certain methodologies are limited by wired setups [14], which poses scalability challenges and diminishes the overall effectiveness and the applicability of the implementation.

Conversely, current approaches that handle heterogeneous networks, like FedProx [9], [15], have not been explored within the context of resource-constrained devices. Commonly employed communication stacks for federated learning, such as Google's Remote Procedure Call (gRPC) [16], Websockets [17], etc., lack adaptability for highly constrained microcontroller-based IoT devices. These devices, operating under stringent resource limitations, require specialized and lightweight communication frameworks. The challenges arise from unique constraints such as limited processing power, communication bandwidth, memory, and energy resources. Addressing these challenges necessitates

the development of tailored protocols that consider resource constraints, and intermittent connectivity. This adaptability is crucial for the practical deployment of distributed learning on edge devices, ensuring that the advantages of FL, such as enhanced privacy and rapid local inference, are not overshadowed by unaddressed scalability and resources concerns. This paradigm shift, leveraged through the TDMiL framework, is essential for extending FL methodologies effectively to IoT devices, to unlock their full potential in this evolving ecosystem.

In light of the challenges outlined above, there arises a compelling need for a dedicated communication stack tailored to meet the constraints of IoT devices, thus enabling on-device learning. The remainder of this paper is organized as follows. First, section II delves into existing research on distributed learning with interconnected microcontrollers, then section III explores available IoT communication protocols. A selection thereof is presented, which is tailored to provide a protocol stack facilitating efficient exchange of model updates within the distributed tiny device landscape. Then, a foundational scenario is presented to illustrate the interplay of different components involved in distributed learning in section IV. Through algorithms thereafter detailed in section V, several distributed learning schemes are introduced, and implemented in section VI. A comparative analysis across different data distributions, model architectures, and varying workloads, demonstrate on-board learning with simple neural networks and transfer learning using deeper neural networks on diverse datasets across multiple IoT devices. Further, section VII and section VIII offer a comprehensive experimental analysis of the communication and computational aspects of experimental results based on these, concluding with discussions on implications and avenues for future research.

II. RELATED WORK

The typical machine learning pipeline feeds data into a model (an artificial neural network), which generates classification or regression outputs that drive further decision-making processes. Machine learning is divided into two sequential phases: the inference phase (using the model) and the learning phase (training the model before use). While the learning phase is more relevant to our present work, key aspects of the inference phase also play a significant role. Additionally, federating the training phase across distinct machines introduces complexities related to networking and distributed systems. Resource constraints on microcontrollers necessitate specific optimizations throughout the process.

A. INFERENCE & MODEL OPTIMIZATION FOR MICROCONTROLLERS

Inference optimization strategies can be coarsely categorized in two levels: the algorithm level and the system scheduling level [18]. At the algorithm level, the main focus is typically model compression techniques, including pruning [4], [19], low-rank decomposition of weights [20], [21], efficient

model architecture [4], [19], and quantization [20], [22] in order to decrease resource consumption of models and inference on-board microcontroller-based devices. On the other hand, at system scheduling level, emphasis is put on the allocation of computation and memory resources [4], [23], or on scheduling between model operators, in prior work such as [24]. In particular, intermediate results required for the training stage (e.g., activations) can be cached in the memory for subsequent gradient calculation. Another domain of prior work such as [25] aims at integrating model optimization and transpilers for diverse model architectures (such as TVM) with common low-power embedded software platforms (such as RIOT [26]) to support a wide range of Microcontroller Units (MCUs).

B. MODEL TRAINING ON MICROCONTROLLERS

On-device training primitives pose an additional optimization and integration challenges on microcontroller-based devices. Prior work on frameworks such as (*TinyTrain* [27], *TTE* [28], *MCUNet* [4], *TinyOL* [29], *POET* [30], etc.) have explored model training on microcontroller-based devices, proposing specialized model architectures [4], [29], optimized operators [28], [30], and sparse update algorithms [27], [28] for efficient on-device training. However, these efforts primarily focused on refining local models with local datasets, lacking consideration for FL. In contrast, this work introduces a general, model-agnostic FL framework, which can seamlessly integrate as a plug-and-play extension for existing on-device training frameworks.

C. DISTRIBUTED LEARNING ON MICROCONTROLLERS

Early work in the area of resource-constrained FL training was presented in [31], using Arduino Nano 33 BLE Sense¹ boards. In a subsequent study [32], a cost-effective transfer learning method was proposed for resource-constrained devices within the FL framework. This method effectively addresses storage challenges during on-device training. The paper emphasizes the need to reduce the number of weight parameters and promote sparsity for improved latency. Additionally, it advocates for the development of a resilient and cost-effective communication protocol to enhance overall system efficiency.

Similarly, TinyFL [14] presents a hybrid primary/subordinate communication protocol for Inter-Integrated Circuit (I²C), designed to improve the efficiency of Microcontroller Units (MCUs) based systems. However, it's essential to note that this protocol is tailored for wired setups, potentially limiting its applicability. Moreover, asynchronous updates may pose challenges within this wired I²C framework.

Recent breakthroughs in TinyML, exemplified by TinyReptile and TinyMetaFed, address unique challenges. Introduced by Ren et al. [33], TinyReptile utilizes meta-learning to customize ML solutions for IoT devices.

¹<https://docs.arduino.cc/hardware/nano-33-ble-sense>

Meta-learning, akin to transfer learning, facilitates fast adaptation to new but similar tasks using limited data samples. It promotes collaborative learning by adjusting neural network initialization. TinyReptile pioneers meta-learning on resource-constrained devices, introduces a novel audio classification dataset, and reduces resource consumption, offering practical insights.

Similarly, TinyMetaFed [34] streamlines collaborative neural network initialization training. It optimizes communication, safeguards privacy, enhances computational efficiency, and ensures robustness through selective transmission of weight parameters. These advancements significantly enhance TinyML, making it more adept for resource-constrained IoT environments.

One observes that much of the aforementioned prior research has primarily focused on computational aspects, [31] brought attention to a bandwidth limitation in communication between MCUs and the server, resulting from the use of a serial port. This limitation leads to increased latency when transferring from the server to the client compared to the client-to-server direction. Likewise, [32] emphasized the critical necessity of enhancing communication costs. The TinyFL approach [14] proposes an alternative network protocol using a hybrid primary-subordinate I²C configuration to address these challenges. However, this wired approach raises significant scalability concerns and cannot effectively leverage cross-device FL with a substantial number of connected tiny devices. Other prior work such as [35] propose efficient model serialization to aiming to decrease network load due to model transfers over the network. On the other hand, TinyMetaFed [34], takes a different approach to alleviate communication costs in FL. Specifically, it suggests transmitting a selective portion of model weights significant for the overall training objective.

We observe that none such prior work addresses the inherent issue of distributed learning asynchronicity, which stems from slow-performing clients i.e. stragglers. This observation highlights a notable gap in the existing literature, which motivates our work.

D. OUR CONTRIBUTIONS

The work we present provides a principled approach to distributed learning in the context of microcontroller-based resource-constrained IoT environments, tackling its fundamental challenges, including constraints stemming from typical wireless communication and the issue of stragglers. Our approach can complement aforementioned meta-learning solutions, such as TinyReptile [33] and TinyMetaFed [34]. More in detail, our contributions are the following:

- **Generic Resource-Constrained Communication for Model & Parameter Transfer:** We introduce a framework utilizing Concise Binary Object Representation (CBOR) and Constrained Application Protocol (CoAP) for efficient exchange of model updates (weights and gradients) from arbitrary model architectures, facili-

tating various distributed learning approaches such as synchronous and asynchronous FL.

- **Handling Asynchronous Resource-Constrained Stragglers:** We introduce strategies for managing asynchronous stragglers with varying workloads in distributed networks, ensuring their effective integration into the learning process.
- **On-Board Training & Transfer Learning:** We design an embedded software platform enabling on-board training and transfer learning on several families of models: simple neural network Multi-Layer Perceptron (MLP) architectures and for deeper networks like LeNet-5, MobileNetV3-M, etc.
- **Comparative Performance Evaluation:** We evaluate the straggler strategies using various standard FL benchmark datasets from the TinyML domain. We experimentally analyze computation, communication costs, latency, and memory footprint on microcontroller-based IoT devices.
- **Open Source Implementation:** We released our code, which builds upon a popular microcontroller-based embedded software platform (RIOT [36]) thus providing a practical basis for future experimental research in this domain.

III. BACKGROUND ON IoT DEVICES & PROTOCOLS

Microcontrollers in smart appliances operate within extremely limited processing capabilities. For instance, as analyzed in [37], low CPU clock frequencies from 10 MHz to 100 MHz and total memory budgets as small as 32 KiB to 512 KiB are not uncommon. Additional hurdles must be faced too: for instance, network links used by these devices are often inadequate for the low latency and high throughput required by most communication protocols. The conventional stack, for instance combining TCP and HTTPS places a significant burden on these devices, making inadequate for resource-constrained environments.

Common FL frameworks like TensorFlow Federated [38] and Flower [39] often rely on Google's Remote Procedure Call (gRPC) [16] as a core element in their communication stack. gRPC, an open-source framework, enables language-agnostic communication between distributed systems using Protocol Buffers (protobuf) [40] for efficient data serialization and transmission. However, deploying gRPC, which entails using TCP and HTTPS, leads to communication bottlenecks on typical resource-constrained devices.

Alternative protocols like WebSockets, which enable lightweight, real-time communication between devices, and Message Queuing Telemetry Transport (MQTT), designed to minimize network bandwidth and power consumption, are commonly considered for resource-constrained environments in FL scenarios. WebSockets facilitate bidirectional, efficient data transfer, while MQTT operates on a publish-subscribe model, particularly suited for scenarios with low bandwidth and unreliable networks. However, it's important to note that even these protocols, while effective, can be

resource-intensive and may not be suitable for severely constrained devices with limited processing capabilities. Hence, other protocols tailored for IoT must be considered. We next survey the landscape of such protocols.

A. IoT COMMUNICATION PROTOCOL LANDSCAPE

Constrained IoT devices require optimizations of all the components of the system, including communication protocols at different layers of the protocol stack. While the IoT communication protocol landscape is still very polymorphic and fragmented, there has been some solidification around IP protocols standards from the Internet Engineering Task Force (IETF), depicted in Figure 1. Let's overview this landscape, keeping in mind potential applicability to distributed learning on microcontrollers.

- **Convergence Layer:** The Convergence Layer begins with the Internet Protocol (IP), either IPv4 or IPv6, which serves as a universal protocol. In order to ensure compatibility with various IoT technologies, there are adaptations made for constrained link-layer connections, such as 802.15.4 through 6LoWPAN [41] which enables to abstract away the variety of MAC layers (for instance low-power radios such as IEEE 802.15.4, BLE, LoRa, etc.).
- **Transport Protocols:** Above IP, one finds transport protocols such as UDP, TCP, and QUIC. DTLS provides security for UDP, while TLS is an option on top of TCP and is mandatory in QUIC. At this level, there are clear tradeoffs between supporting very constrained devices and providing convenient features. For instance, TCP automatically manages multiple packets in parallel, while QUIC (on top of UDP) allows for multiplexing of streams. These features, however, require buffering (RAM) and a more complex state machine (CPU).

- **Web Transfer Protocols:** On top and along the transport protocols, web transfer protocols are used for named data exchanges. While on the less resource-constrained machines protocols such as HTTP (secured with TLS) or MQTT are used, on microcontroller-based IoT devices more lightweight equivalents are used such as, respectively, CoAP (secured with OSCORE or DTLS) and MQTT-SN. Work such as [42] suggests that in scenarios with highly dynamic clients, especially during asynchronous events, the recommendation is to utilize CoAP Pub/Sub rather than Message Queuing Telemetry Transport-Sensor Networks (MQTT-SN), for applications handling large payloads in environments prone to high packet loss, which matches our scenario. This preference is rooted in CoAP's inherent support for application layer fragmentation, effectively preventing message drops caused by exceeding the allowed number of retransmissions.
- **Management Protocols:** Above web transfer protocols, another layer of protocols may be used to provide remote management of devices, over the network. While the less resource-constrained machines use protocols such as gRPC, Websockets (or NetConf, SNMP, etc), on microcontroller-based IoT devices more lightweight alternatives are used such as Light-weight Machine-to-Machine (LwM2M). For instance, LwM2M can use CoAP as a transfer protocol and extend its capabilities to unify management tasks, offering a standardized, foundational IoT device management framework.
- **Data Representation & Serialization Tools:** Actual application data transferred from one machine to another (potentially based on different hardware!) must be represented and serialized in a convenient and interoperable format. While the less resource-constrained machines use serialization schemes such as JSON and XML, on microcontroller-based IoT devices more lightweight alternatives are used to incur fewer bits-over-the-air, such as with Concise Binary Object Representation (CBOR) binary formats or protobuf.

We now transition towards solution space, considering the following simple yet generic scenario.

IV. SIMPLE SCENARIO WITH DISTRIBUTED LEARNING

The scenario, we will study for the remainder of the paper involves a PC server without resource constraints (i.e. with typical computer with several order of magnitude more resources compared to a microcontroller), which is connected via an arbitrary network to N microcontroller-based machine learning clients, as depicted in Figure 2.

The FL server (the PC) takes on both the roles of orchestrator (registering clients, receiving/sending data to/from clients, etc.) and aggregator (combining models from multiple clients into a single aggregated model). Note that the roles orchestrator and aggregator can either be split across separate

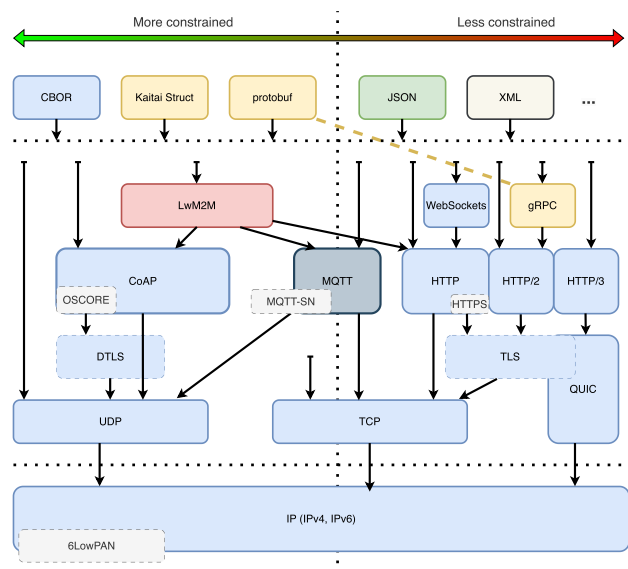


FIGURE 1. Communication protocol landscape for IoT.

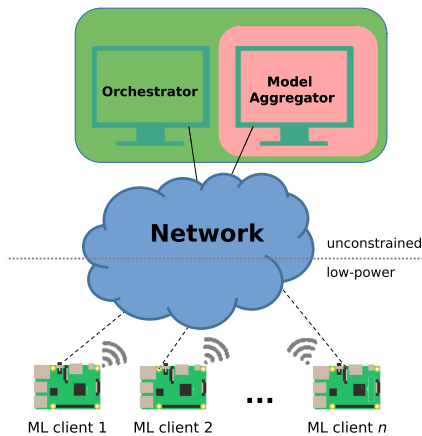


FIGURE 2. Different Distributed Configurations.

machines (e.g., for security/privacy purposes) or combined on the same machine.

Each client runs on microcontroller-based hardware, with a tiny memory budget (e.g., 512 kB), connected via heterogeneous network segments including one or more low-power wireless lossy links (e.g., IEEE 802.15.4, BLE, etc.).

Training data is either pre-provisioned on each client or obtained on-the-fly Out of Band (OOB) on each client. The learning process can be offline or online. For the sake of simplicity, we only showcase the approach using offline recorded datasets.

A. ASSUMPTIONS & LIMITS OF THE SCENARIO

In our scenario, we consider that individual network paths (including their low-power segments) from a client to the orchestrator are not significantly impacted by external, internal or cross-interference or side-traffic.

- *Basic network & node fault tolerance:* We assume network and node fault tolerance mechanisms typically at work within the IoT network stack can handle most cases. Such mechanisms include for e.g., CoAP confirmable message retransmissions at the application layer (see [43] section II-A), while complementary mechanisms are provided in the routing protocol and in the lower layers of the stack (e.g., IEEE 802.15.4 retransmissions [44]).

Indeed, prior work (see [45] Table 4) confirms that *large* network transfers such as firmware updates of size ≈ 100 kB are transmitted without significant issues over a comparable low-power link (IEEE 802.15.4 using the 6LoWPAN and CoAP). Other related work (see [25] Table 3) indicates that models in typical TinyML model zoos, incur comparable transfer load over the network, i.e., ≈ 100 kB. Hence, while significant follow-up work should be dedicated to develop additional mechanisms to optimize specifically for FL the handling of network disruptions and node failures, our work on the above

basic scenario should serve as solid base for such follow-up work.

- *Number of clients:* In the scenario we consider, and in our experiments below, we tackle only a handful of clients. However, since we assume that clients' data transmissions do not significantly interfere with one another, the bottleneck is essentially shifted to the orchestrator's (unconstrained) link to the network and its (unconstrained) processing power. Therefore, we do not further address this scalability issue in this paper. Conversely, in cases where significant external interference or cross-interference are observed for data transmission from clients to the orchestrator, specific additional mechanisms would be required to handle these appropriately. Nevertheless, our work on the simple scenario we describe should serve as a solid base for such follow-up work.

V. SYSTEM DESIGN

A. COMMUNICATION STACK FOR DISTRIBUTED LEARNING ON MICROCONTROLLERS

We design our communication stack on top of CoAP and CBOR for serialization, which offers specific advantages in addressing challenges posed by resource-constrained environments. We leverage CoAP capabilities for reliable asynchronous communication and blockwise transfer, which cater both for dynamic settings and large payloads in face small maximum packet size, and packet loss typical in IoT. Additionally, we leverage CoAP support for Datagram Transport Layer Security (DTLS), securing efficiently IoT client-server communications. We chose CBOR over protobuf for serialization because prior work such as [35] hints that, for arbitrary model sizes, best-case CBOR outperforms best-case protobuf, which makes it favorable for optimal data representation. The combination of CBOR and CoAP thus streamlines IoT client-server data exchanges, and caters to the unique needs of communication of heterogeneous, low-power network links. The stack further allows seamless integration with established FL frameworks like Flower [39], TensorFlow Federated [38], etc. The shift we propose here stems from our realization that these frameworks, in their current configurations, primarily cater to unconstrained server setups. In contrast, our framework takes a progressive step forward and opens the door towards accommodating even more diverse configurations, such as (in future work) hosting the orchestrator/aggregator on constrained devices too, or designing and implementing model aggregation without a central agent at all (e.g. Gossip Learning).

B. MODEL SERIALIZATION

As the orchestrator is in full control of the distributed FL process, it requires multiple types of messages exchanged to fully control the learning process on the clients. We define multiple CBOR encoded messages for this purpose. By using CBOR as message serialization format the message size

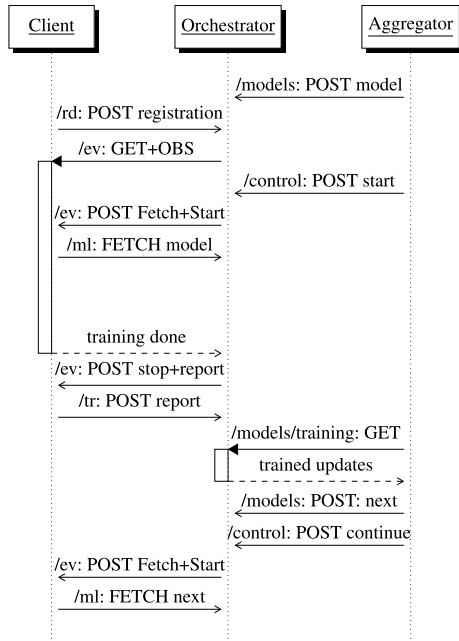


FIGURE 3. Start-up, provision and training network call sequence.

is minimal. CBOR has a comparable serialization when compared with protobuf, however CBOR is used frequently in the IETF and integrates well with other protocols such as CoAP.

As part of the communication stack, we design a number of messages passed between clients and the central server. A key part of this is the efficient serialization of the ML models. Part of the requirements is that the message must allow for partial updates of the ML model, without the need to transfer the full ML models over the network.

To achieve this, we split a ML model into a logical set of operators used within the model. Each operator has one or more sets of parameters, or weights. Our serialization contains both the set of parameters and the operators, with the operators containing a reference to which parameters are used by the operator. Note that parameters can be shared between different operators where possible. A full model update contains the full set of parameters and operators in the model, while a partial update only transfers the set of operators and parameters required for the update useful in case of transfer learning task.

The ML models contain an Universally Unique Identifier (UUID) as handle, used in the communication messages. Together with the layer and parameter identifiers, a client is able to request an individual set of parameters from the server for the purpose of partial model updates.

C. COMPONENT INTERACTION

The CoAP interaction between the clients and the orchestrator using CoAP leverages a mixed strategy with both the GET + Observe mechanism and FETCH-type [46] requests to minimize the data overhead between the clients and the orchestrator as described in Figure 3.

On start-up, the clients register themselves using the CoAP resource directory protocol [47] with the orchestrator. This provides the orchestrator with a full view of which clients are available for the FL process.

After a client has registered, the orchestrator establishes a bidirectional message channel with the client based on a CoAP GET + Observe [48] request and POST requests to a /ev endpoint on the client. Via this channel, a client can report an event to the orchestrator as one of the responses to the GET + Observe request. The orchestrator can send events to the client using a regular CoAP POST request. This channel is used as a bidirectional event and RPC channel, specifically for short commands and events in our design.

Via the bidirectional message channel, the clients can for example be instructed to fetch a new model from the orchestrator. Clients download the model via CoAP FETCH requests at the /ml endpoint provided by the orchestrator, with the payload in the initial FETCH request specifying which model and, when needed, which operators are requested. The models are transferred via the block-wise mechanism by CoAP to keep the network packets within the limitations of the typical constrained network links such as IEEE 802.15.4 networks.

The orchestrator manages the FL clients on behalf of the aggregator. The aggregator provides the orchestrator with new model updates and retrieves the training meta-data from the orchestrator. While the aggregator is aware of successive model updates based on rounds, the orchestrator and clients do not need to be aware of relations between models provided and operate on models based on their UUID.

D. DISTRIBUTED LEARNING SCHEMES

In contrast to centralized learning, where clients send their raw local data directly to the server, compromising data privacy, our work emphasizes the exchange of trained model parameters instead of raw datasets. We consider the following approaches for distributed learning:

- **Algorithm 1: Synchronous - FedAvg approach [7]).** Clients train local models using their datasets and send model updates to the central server. The server aggregates these updates and returns the aggregated model. However, updates from slow-performing clients (stragglers) are discarded.
- **Algorithm 2: Asynchronous - partial work approach).** Similar to Algorithm 1, but here, updates from stragglers are incorporated. The aggregator integrates stragglers proportionally to the fraction of the learning completed.

The primary distinction between the algorithms lies in how they address stragglers during the aggregation process, as indicated by the yellow and green highlights in algorithm 1 and algorithm 2, respectively.

Algorithm 1 Synchronous in subsection V-D**Initialization (by Model Aggregator):**

Orchestrator receives a new global model from the aggregator, identified as W_{global} .
 Write serialized model and metadata for round 0 on the /models endpoint using an HTTP POST request.
 Submit configuration, including the model identifier `global` and `total_rounds`, to all clients.

Client-Side Training Loop:

foreach *participating client i in the federated network* **do**

Initialize local model parameters W_{local_i} for the round with W_{global} , as configured by the orchestrator, through a CoAP FETCH request on /ml.
 Train and update W_{local_i} based on local data for E epochs using a local optimizer.
 Write a local update to the /tr endpoint for round r along with metadata containing `num_examples` and other metrics using a POST request.

Server-Side Aggregation:

At the central aggregator:

A possible criterion: **foreach** *completed work of local clients in the queue* **do**

If the desired % of clients finish local training within a specific round, GET to retrieve W_{local_i} and metadata from these clients by accessing the endpoint
 /models/<global>/training/<client_id>.
 Aggregator can perform the aggregation of these clients and discard the remaining clients.
 Calculate the weighted average of client model parameters to obtain the new global model parameters W_{global} based on Equation 1 with ϵ as one.

When to Trigger Aggregation:

Based on one of two possible criteria defined in subsection V-E.

Algorithm 2 Asynchronous in subsection V-D**Initialization (by Model Aggregator):**

Orchestrator receives a new global model from aggregator identified as W_{global} .
 Write serialized model and metadata for round 0 on /models endpoint using HTTP POST request.
 Submit configuration including model identifier `global` and `total_rounds` to all clients.
 Wait for sufficient messages from clients that their training is completed.
 Submit messages to all clients to stop training and report the training data.

Client-Side Training Loop:

foreach *participating client i in the federated network* **do**

Carry out usual training process as described in synchronous setting until the stop message is received from the orchestrator.
if *stopped* **then**
 Make a POST request with the current update of the model along with
`num_of_completed_epochs` to /tr endpoint on the orchestrator.
 Wait for new instructions from the orchestrator.

Server-Side Aggregation:

At the central aggregator:

A possible criterion: **foreach** *completed work of local clients in the queue* **do**

If desired % of clients finish local training within a specific round, the aggregator can perform the aggregation using Equation 1.
 Consider partial contributions of remaining clients by multiplying with their ratio of work completed, i.e., $\epsilon = \frac{e}{E}$ completed epochs over assigned epochs initially.

E. MODEL AGGREGATION SCHEMES

For triggering aggregation in an FL setting, two possibilities are considered

- **Possibility 1 (Queue-based Criterion):** The server maintains a queue for clients who have completed local training from the current round. When a desired fraction of the total participating clients completes their training, aggregation is triggered. Contributions from the remaining clients are not considered in this scenario.
- **Possibility 2 (Client-based Criterion):** Aggregation is triggered when updates from specific clients (based on `client_id`) are present in the queue. This approach allows for a more targeted aggregation based on the status of specific clients.

VI. TDMiL IMPLEMENTATION

Our implementation consists of systems aspects on the one hand, and on the other hand, on data sets and data handling aspects.

A. SYSTEM

Our implementation consists of multiple components spread across different systems.

1) MODEL AGGREGATOR

The model aggregator is a central entity responsible for combining locally trained models from participating clients. One common aggregation method is weighted averaging applied in [7], where the updates from each client are weighted based on the ratio of local data points to the total data points across all clients.

In asynchronous setups, this weighting considers the fraction of local points used in training relative to the overall points in the FL round, ensuring that even partially completed work from slower clients is incorporated, rather than discarding their contributions entirely. This adaptive approach addresses variations in client device speeds, promoting efficient model aggregation in FL scenarios. The global update weight (W_{global}) is computed using the formula:

$$W_{\text{global}} = \frac{\sum (n_i \cdot \epsilon_i \cdot W_{\text{local}_i})}{\sum n_i} \quad (1)$$

where n_i , $\epsilon_i = \frac{e}{E}$ denote the number of data samples and completed epochs over assigned epochs by client i respectively. W_{global} , W_{local_i} denote global model parameters and local model parameters for client i respectively.

2) ORCHESTRATOR

Our custom orchestrator, “CoAPerator”, is a key actor in our distributed learning setup. It manages CoAP-enabled client devices and provides the interface for the Model Aggregator to facilitate the FL algorithms. Developed with FastAPI [49] and aiocoap [50], CoAPerator is a Python-based application that seamlessly integrates robust functionality, providing streamlined device management within the CoAP ecosystem.

It allows clients to register themselves with the orchestrator, through a CoAP resource directory service. From there on the orchestrator sets up a bidirectional communication channel over CoAP to manage the registered clients by providing RPC-style commands to the clients. Furthermore, it provides the endpoints for providing ML models to the clients. As previously mentioned the orchestrator exposes these services via CoAP.

Interaction with the orchestrator by administrators and by the model aggregator does not occur via the constrained network used by the clients. To ease interaction with this interface, it is provided via a REST-like API using HTTP and JSON. This interface is used to upload new models to the orchestrator, for example by the Model Aggregator when a new round is to be started.

3) FL CLIENTS

The FL clients run on constrained devices with firmware consisting of: RIOT [36] as an embedded operating system; An IPv6-enabled network stack; A CoAP stack; and the necessary primitives for FL and ML.

The clients are pre-provisioned with the location of the orchestrator and on start-up contact the orchestrator for registration. After registration, the clients expect a channel to be established, and initiated by the orchestrator in response to the completed registration process.

The clients are able to update the parameter and configuration of the FL process on the fly, based on instructions from the orchestrator. However, updating the structure of the deployed models is not possible without resorting to a firmware update.

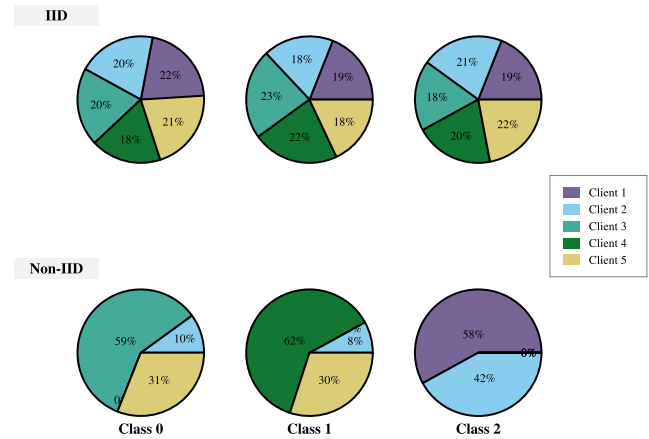


FIGURE 4. Clients distribution (IID vs Non-IID) on iris dataset.

B. DATA HANDLING

Tabular Dataset – We implemented min-max scaling to normalize features within the range of 0 to 1. Given the relatively small size of the original tabular datasets (iris [51], wine [52], and breast cancer [53]) shown in Table 1, comprising a few hundred data samples, we employed a data augmentation technique. This involved applying Gaussian noise with a mean of zero and a standard deviation ranging from 0.05 to 0.25, effectively expanding the size of the original dataset.

The augmented dataset was subsequently utilized for both training and validation purposes, employing an 80-20% split. This divided dataset was then further partitioned among five clients. The data distribution among these clients could either be independent and identically distributed (IID) or non-IID, and we will showcase results for both scenarios. The original dataset remained unchanged and was utilized as the test set for the final evaluation (presented in Table 6) of the model trained in a distributed manner.

To analyze the impact of varying levels of diversity across clients, we employed the widely recognized technique of Latent Dirichlet Allocation (LDA) [54]. By adjusting the hyperparameter α , we could control the diversity among clients, thus creating local datasets that maximally capture real-world variations. Specifically, we set α to a low value, 10^{-6} , to depict an extreme scenario where diversity across clients is high. This approach results in clients receiving data points with imbalanced label classes in the non-IID case, as opposed to balanced labels in the IID case. This is evident when examining the class distributions across clients within our experiments: in the IID case, the height of the bars in histograms is relatively comparable across classes, while in the non-IID case, the heights vary significantly, indicating statistical heterogeneity among clients. This variation is illustrated in Figure 4, Figure 5, and Figure 6. The data used in the experiments can be accessed through our code repository.

MNIST Dataset – This dataset is a widely used collection of handwritten digits (28×28 pixel, grayscale) that serves

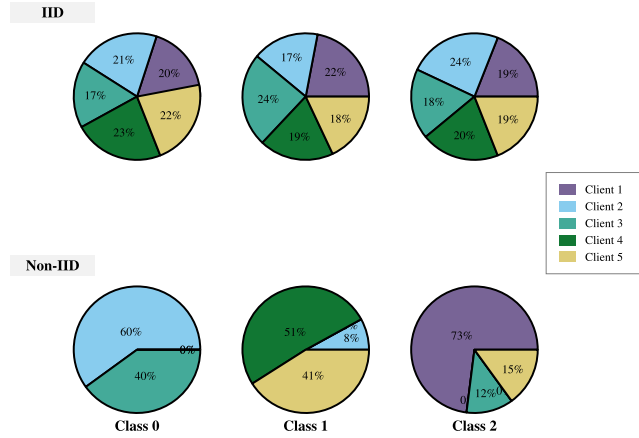


FIGURE 5. Clients distribution (IID vs Non-IID) on wine dataset.

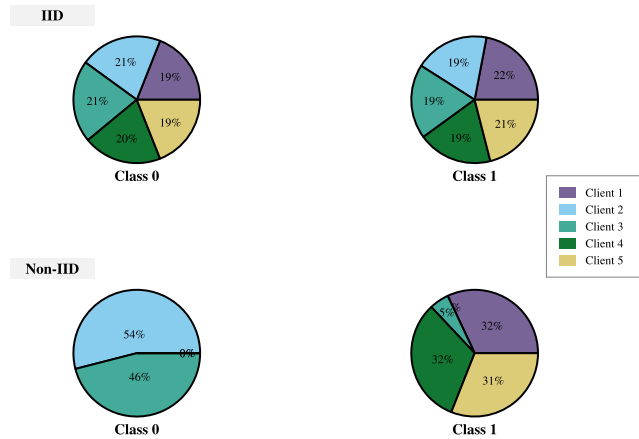


FIGURE 6. Clients distribution (IID vs Non-IID) on cancer dataset.

as a benchmark in the field of ML and computer vision. We standardized the grayscale features, originally ranging from 0 to 255, to a normalized scale of 0 to 1 through min-max normalization. Each client received a pre-allocation of 125 samples for both local training and validation, divided in an 80:20 ratio. The data distribution among clients was IID. We employed a test set of 100 samples to assess the performance of the global model.

TABLE 1. Summary of datasets.

Dataset	Features	Classes	Samples
Iris Flowers [51]	4	3	150
Wine [52]	13	3	178
Breast Cancer [53]	30	2	569
MNIST [55]	$28 \times 28 \times 1$	10	725

VII. EXPERIMENTAL SETUP

A. HARDWARE SETUP

CoAPerator, our Python-based orchestrator, runs on a basic laptop running Linux. Our TDMiL clients are tested running on several commercially available boards such as the

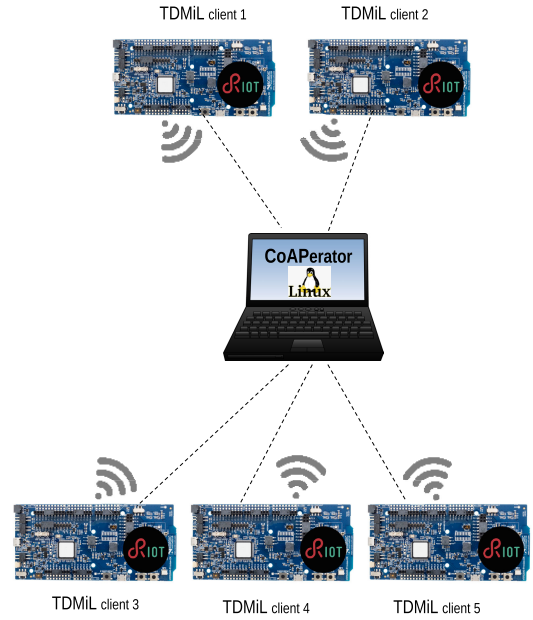


FIGURE 7. Experimental setup.

nrf52840 development kit (nrf52840dk), Raspberry Pi Pico (rpi-pico) and STM32F746 discovery board (stm32f746g-disco). For instance, the nrf52840dk is based on an Arm Cortex-M4 MCU running at 64 MHz, with a total memory budget of 1 MB Flash (ROM) and 256 KB RAM. The STM32F746 board is equipped with a powerful MCU - Cortex-M7 at 216 MHz, while the raspberry pi pico holds a more energy efficient Cortex-M0+ MCU at 133 MHz.

Note on generality & reproducibility: as TDMiL is based on RIOT [36] and CoAPerator on Linux, the code we provide is widely applicable beyond the specific hardware we indicated above. In particular, most boards supported by RIOT (hundreds of different boards) can run a TDMiL client.

B. NETWORK SETUP

We test our implementations interconnected as such: one CoAPerator instance is connected in hub-and-spoke fashion (star topology) to five TDMiL clients as shown in Figure 7.

We test our different schemes for (i) distributed learning and (ii) model aggregation on top of this setup.

Note on generality & reproducibility: our experiments can be reproduced in two easy ways. The first way is using five RIOT native instances [56] and tap interfaces together with ZEP (Zigbee Encapsulation Protocol, used to tunnel 6LoWPAN over UDP). This topology can be emulated and tested on any Linux machine. Another way is to actually book five nrf52840dk boards co-located on the open-access testbed IoT-lab [57], [58] and to flash them with TDMiL client firmware.

C. SOFTWARE SETUP

We used RIOT version 2023.10, with compiler a toolchain based on GCC 12.2.

D. METHODOLOGY FOR STRAGGLERS

To simulate stragglers, deliberate delays were introduced between local epochs, leading the orchestrator to potentially direct clients to cease local training. In this context, incomplete work from paused clients undergoes one of two variants, outlined in subsection V-D: either it is discarded entirely (Algorithm 1), or it is effectively weighed by the aggregator (Algorithm 2).

1) IMPLEMENTED MODELS & ON-BOARD TRAINING

In order to test our design on relevant models, we picked the one that had a few “toy” models based on a basic Multi-Layer Perceptron (MLP) architecture, and on the other hand, LeNet5 and MobileNetV3-M models, which are more complex models used in TinyML literature (e.g., see model zoo used in [25]).

- **MLP models:** We have implemented a three-layer MLP with a variable number of neurons tailored to different tabular datasets with input, hidden and output dimensions as [4, 2, 3], [13, 10, 3] and [30, 10, 2] for the respective iris, wine, and breast cancer datasets. For the classification tasks with a cross-entropy loss, the number of trainable parameters for the respective datasets are as follows: 19, 173, and 332 parameters. In this straightforward configuration, we have manually crafted the gradient computation. The model’s weights and biases are updated using a stochastic gradient descent (SGD) approach, and the computations are performed in an Eigen [59] C++ based library. For a more in-depth understanding of the implementation, the code can be accessed through the provided link.
- **LeNet5 [60]:** This representative early model with about 44.6K parameters specialized in classifying hand-written digits is a simple convolutional neural network with two convolutional layers and three fully connected layers. It consumes 28×28 grayscale image and produces likelihoods over 10 digit classes.
- **MobileNetV3-M:** This model is based on MobileNetV3 [61], an advanced backbone focuses on optimizing mobile and embedded vision applications efficiently. It leverages depth-wise separable convolution to achieve significantly higher efficiency compared to standard convolution (roughly 8 to 9 times less computation) with only a slight decrease in accuracy. In this study, we tailored it to mirror the input, output and classifier structures of LeNet5, yielding approximately 112.8K parameters. The classifier contains 84 and 10 neuron units in the last two fully connected layers, respectively. This customization of MobileNetV3 decreased drastically the resource consumption of tiny devices with slightly degradation of model accuracy on MNIST dataset. For the sake of clarity, we use the name “*MobileNetV3-M*” for this tailored model in this work.

Transfer learning setup – Both LeNet5 and MobileNetV3-M were pre-trained on the MNIST dataset, then transpiled

and deployed on constrained devices using U-TOE [25]. For purpose of experiments, only the weights of the final layer (classifier) were reinitialized on uniform distribution and federated fine-tuned on devices. We also used cross-entropy for loss calculation and SGD optimizer with a learning rate of 0.01 and a weight decay of 0.01 for model training.

VIII. EXPERIMENTAL RESULTS

A. RESOURCE CONSUMPTION ON DEVICE

We measured storage (FLASH) and memory (RAM) use, as well as compute latency for each model in Table 2, Table 3 and Table 4. Peculiarly, the MLP model consumed a disproportionate amount of memory and more storage than expected based on the number of model parameters. As it turns out, dynamic-sized types from the Eigen library pre-allocate relatively large vectors and matrices. Further optimizations are needed here for low-end MCUs.

While the models exhibit similar memory and storage usage across the three MCUs, the compute latency varies significantly due to differences in core frequencies and instruction sets. The stm32f746g-disco board (Cortex-M7), with its advanced architecture, and highest core frequency, achieved the best compute performance on all models. One notable outlier is the RP2040-based (Cortex-M0+) Raspberry Pi Pico board. Despite having a higher core clock frequency than the nrf52840dk board (133 MHz vs. 64 MHz), its compute latency is approximately $10\times$ slower on LeNet5 and MobileNetV3-M. This is due to the RP2040 microcontroller lacking a hardware floating-point unit (FPU), leading to a significant performance drop when testing unquantized models using floating points.

TABLE 2. Storage & memory use on the Nordic nrf52840dk board (Cortex-M4 microcontroller).

Entity	Memory (kB)	Storage (kB)	Compute Latency (ms)
MCU HW available	256.0	1024.0	-
Network Stack firmware	13.7	35.1	-
MLP (19)	57.1	138.4	0.35
LeNet5 (44.6K)	37.0	185.0	55.2
- Final Layer	3.4	3.4	-
MobileNetV3-M (112.8K)	39.0	480.6	92.3
- Final Layer	3.4	3.4	-

The numbers of model parameters are presented alongside model names.

B. FL RESULTS WITHOUT STRAGGLERS

1) BITS OVER-THE-AIR

We measured the number of bytes transiting over the network with CoAP block-wise transfer, between clients and the orchestrator. For each round, each client fetches new model weights back from the server after model aggregation. To no surprise, we observe in Figure 8 the network transfer costs growing linearly w.r.t. rounds and clients. Table 5 analyses more precisely network traffic structure for each model transfer over the network. We notice that *control traffic* overhead is significant for large models: for instance, with

TABLE 3. Storage & memory use on the ST Microelectronics stm32f746g-disco board (Cortex-M7 microcontroller).

Entity	Memory (kB)	Storage (kB)	Compute Latency (ms)
MCU HW available	340.0	1024.0	-
Network Stack firmware	7.9	30.6	-
MLP (19)	58.7	148.0	0.15
LeNet5 (44.6K)	37.2	182.3	25.0
- Final Layer	3.4	3.4	-
MobileNetV3-M (112.8K)	39.0	477.3	59.2
- Final Layer	3.4	3.4	-

The numbers of model parameters are presented alongside model names.

TABLE 4. Storage & memory use on the Raspberry Pi Pico board (RP2040 microcontroller).

Entity	Memory (kB)	Storage (kB)	Compute Latency (ms)
MCU HW available	264.0	2048.0	-
Network Stack firmware	4.9	21.0	-
MLP (19)	58.8	150.2	0.30
LeNet5 (44.6K)	37.6	182.9	529.0
- Final Layer	3.4	3.4	-
MobileNetV3-M (112.8K)	38.7	426.1	724.1
- Final Layer	3.4	3.4	-

The numbers of model parameters are presented alongside model names.

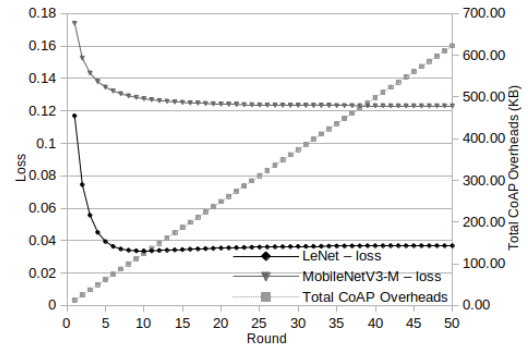
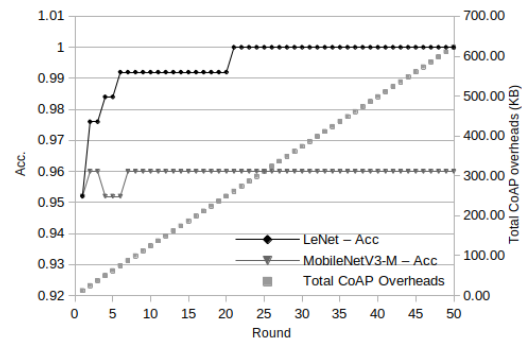
LeNet and MobileNetV3-M, the total traffic for fetching the model update is $1246 + 3977 = 5223$ B (see last row, first column in Table 5). However, the *goodput* is only ≈ 3.3 kB, which corresponds to the size of the layer being transmitted. The difference yields ≈ 1.8 kB of control traffic overhead (+55%). One approach to decrease control overhead for large models is to increase the CoAP block size (64 B in our experiments).

2) MODEL PERFORMANCE

We averaged the validation losses and Top-1 accuracy for the five clients at the end of each round in Figure 8 and Figure 9. With IID distributed training data, both model converged and achieved over 96% Top-1 accuracy. On the other hand, LeNet5 outperforms MobileNetV3-M by 4% accuracy, although it is a smaller model (50% less parameters). This suggests an over-fitting issue, which should be addressed via additional regularization measures for MobileNetV3-M in this case.

3) KEY TAKEAWAY

We also observe that, though both models converged well under our FL scheme, they stagnated around 20 rounds. In order to mitigate model over-fitting and avoid higher resource consumption (computing costs, network transfer costs, etc.) a strategy such as *early stopping* [62] can be employed which stops FL training earlier upon detecting such stagnation.

**FIGURE 8.** Average validation loss of LeNet5 and MobileNetV3-M, with Algorithm 1 IID, on MNIST dataset.**FIGURE 9.** Average validation Top-1 accuracy of LeNet5 and MobileNetV3-M, with Algorithm 1 IID, on MNIST dataset.

C. FL RESULTS WITH STRAGGLERS

We now compare Algorithm 1 and Algorithm 2 when facing stragglers as specified in Section VII-D.

1) BITS OVER-THE-AIR

The primary payload, encoded in CBOR, comprises mostly float-represented model parameters during over-the-air transmission. Control data makes up the remaining portion (Figure 10), transmitted via CoAP in 64-byte blocks. FETCH requests are employed to request models, and the corresponding responses include CBOR encoded payloads, with the associated costs elaborated in Table 5. We conducted a controlled experiment involving two stragglers in each round. Algorithm 1 minimizes communication overhead by leveraging the orchestrator's ability to execute fetch requests exclusively on clients that have completed local training, as determined by the status of all clients. This results in a notable reduction in the number of fetch requests, decreasing from 5 to 3 in comparison to Algorithm 2. The total number of requests and responses between all clients and the orchestrator remains constant at 5 for both algorithms. Notably, Algorithm 1 achieves a lower exchanged bits per round by strategically excluding stragglers from contributing their model updates for aggregation.

2) MODEL PERFORMANCE

Both algorithms yield similar results in average metrics, computed by weighting based on the fraction of local client data during validation and testing, as depicted by the red (Algorithm 1) and purple (Algorithm 2) curves in Figure 11 to Figure 14, and further detailed in Table 6 for the IID setting. For all models shown in these figures (MLP-based models), performance peaks after just 15 rounds of training for IID data distribution. This indicates that even discarding the contributions of slow-performing clients (stragglers) does not adversely affect the performance on average metrics. On the other hand, including the contributions from stragglers adds little to the overall metric since the IID distribution is statistically similar across clients.

However, in the case of Non-IID data, Algorithm 2 (black curves) outperforms Algorithm 1 (blue curves), demonstrating stable convergence by intelligently aggregating the contributions of stragglers. It is evident that Algorithm 1 faces challenges when dealing with a Non-IID distribution, as it fails to converge, resulting in unsatisfactory outcomes. This is because discarding updates from slow clients with statistically important data points can lead to poor performance and the high fluctuations observed in the blue curve for Algorithm 1 under Non-IID data.

3) KEY TAKEAWAY

Algorithm 1 achieves a lighter communication overhead compared to Algorithm 2. However, this advantage comes with a trade-off in performance, particularly evident in Non-IID distribution scenarios, as illustrated in Table 5 and Table 6.

TABLE 5. Transfer costs per round (various models & algorithms).

Model	Component	Exchange (B)	Algo1 (kB) ×8	Algo2 (kB) ×10
MLP (Iris)	Request	64	0.50	0.63
	Response	162	1.27	1.58
MLP (Wine)	Request	284	2.22	2.77
	Response	866	6.77	8.46
MLP (Breast Cancer)	Request	510	3.98	4.98
	Response	1599	12.49	15.62
LeNet / MobileNetV3-M Final Layer	Request	1246	9.73	12.17
	Response	3977	31.07	38.84

IX. DISCUSSION & FUTURE WORK

One significant limitation of our system thus far is the single point of failure presented by the FL server. This centralized architecture poses risks to system reliability and scalability, particularly in dynamic IoT environments. To mitigate this issue, we propose enabling model aggregation without a central agent, in a peer-to-peer fashion, such as Gossip Learning (GL). This approach would distribute the aggregation process across multiple nodes, enhancing system resilience and potentially improving network efficiency. Notably,

TABLE 6. Final accuracy & loss on different datasets.

Dataset	Configuration	Final Accuracy	Final Loss
Iris	Algorithm 1 IID	0.98	13.5
	Algorithm 1 Non-IID	0.667	188.3
	Algorithm 2 IID	0.98	11.9
	Algorithm 2 Non-IID	0.967	19.8
Wine	Algorithm 1 IID	0.988	14.1
	Algorithm 1 Non-IID	0.887	46.1
	Algorithm 2 IID	0.988	15.2
	Algorithm 2 Non-IID	0.938	34.5
Breast Cancer	Algorithm 1 IID	0.971	55.2
	Algorithm 1 Non-IID	0.908	111.0
	Algorithm 2 IID	0.975	58.2
	Algorithm 2 Non-IID	0.956	73.3

while our implementation of CoAPerator on Linux is not suitable for microcontroller-based devices due to resource constraints, our implementation of TDMiL clients on RIOT provides an excellent foundation for further development. This RIOT-based implementation can be leveraged to design, implement, and comparatively evaluate fully decentralized, peer-to-peer model aggregation schemes on various wireless microcontroller-based IoT devices. Such an evaluation would offer valuable insights into the performance and scalability of decentralized learning in resource-constrained environments, focusing on the challenges specific to microcontroller-based systems.

Another critical limitation of TDMiL is its potential vulnerability in terms of privacy. While the most sensitive information (raw data) is indeed protected by not leaving the devices on which it is produced and trained, the models themselves and their parameters (e.g., the weights) also constitute sensitive information that may need to remain confidential. Compromised model aggregators could potentially leak the models (or parameters) provided by the clients, leading to unauthorized access to information about the training data. Currently, TDMiL does not provide any mechanism to ensure such confidentiality, which is a significant concern in many real-world applications, especially those involving sensitive or proprietary data.

In future work, we plan to focus on privacy-enhancing techniques for TDMiL and CoAPerator to address these concerns. Specifically, we aim to augment the framework with a robust layer for privacy protection based on Differential Privacy (DP) techniques and secure aggregation strategies using homomorphic encryption. DP would allow us to add controlled noise to the model updates, providing a provable privacy guarantee while maintaining useful learning outcomes. Homomorphic encryption, on the other hand, would enable computations on encrypted data, ensuring that the aggregator can combine model updates without accessing the raw parameters. Both the design and practical implementation of such a layer present significant challenges on microcontroller-based devices, given their limited computational resources. Nevertheless, addressing these challenges

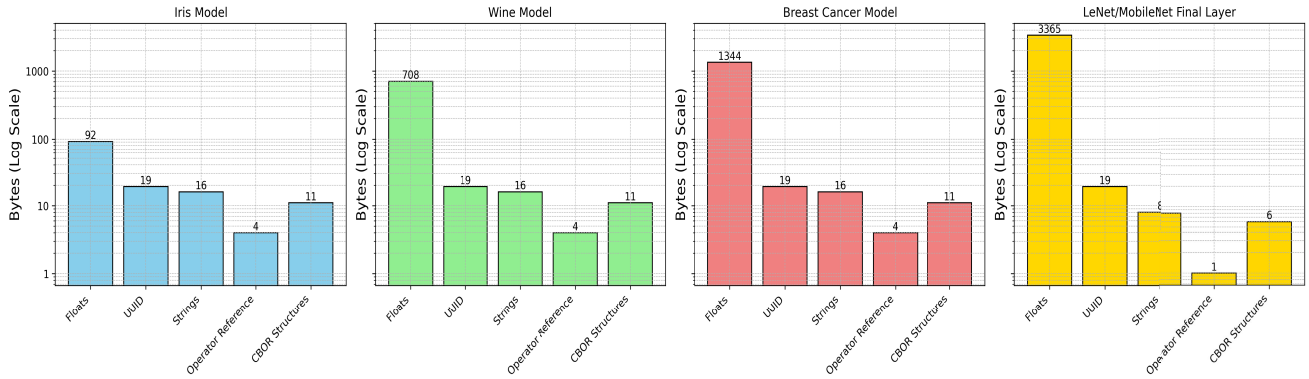


FIGURE 10. CBOR Payloads with total sizes 142 B, 758 B, 1395 B, 3399 B respectively.

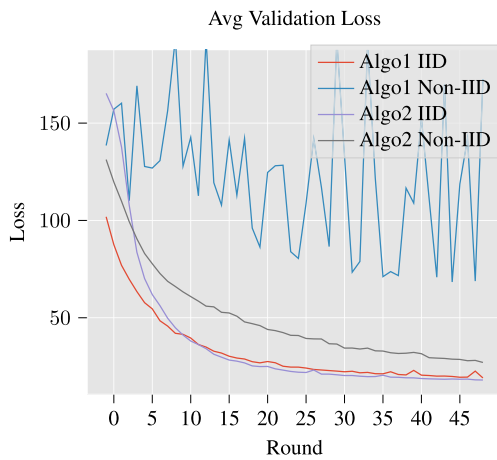


FIGURE 11. Validation loss with iris data.

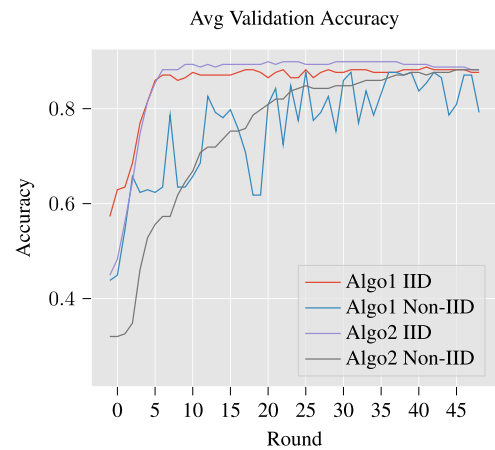


FIGURE 13. Validation accuracy with wine data.

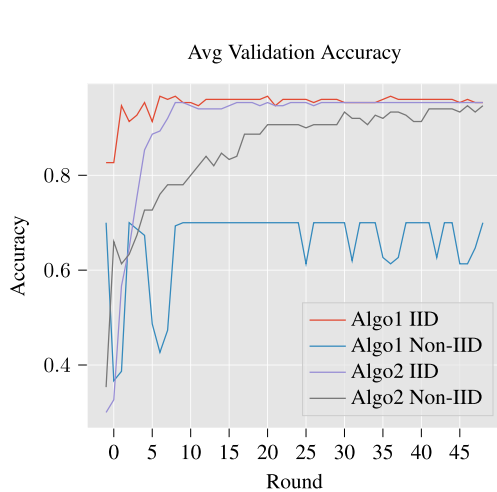


FIGURE 12. Validation accuracy with iris data.

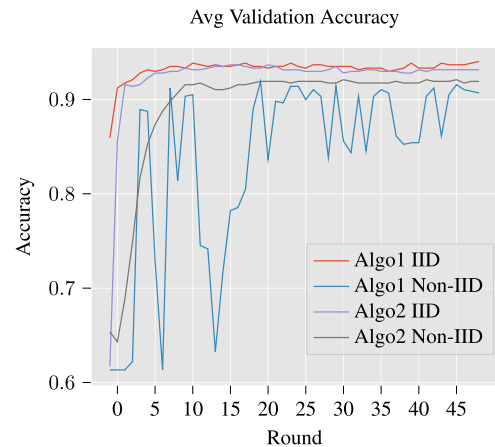


FIGURE 14. Validation accuracy with breast cancer data.

is crucial in this context to maintain the confidentiality of the models and broaden the applicability of TDMiL to privacy-sensitive domains. Successful implementation of these privacy-enhancing techniques could potentially open up new use cases for federated learning in IoT environments and

contribute to the broader field of privacy-preserving machine learning on edge devices.

X. CONCLUSION

The concurrent acceleration of AI advancements and IoT proliferation presents a complex landscape of intersecting yet sometimes conflicting requirements. Addressing these challenges, we introduced TDMiL and CoPerator,

integral components of an innovative framework meticulously designed for distributed training and transfer learning. Our solution is specifically engineered to operate efficiently within the constraints of low-bandwidth, lossy wireless networks, facilitating seamless interconnection among resource-limited devices built on diverse microcontroller architectures.

At the core of our research lies a critical challenge inherent to fleets of microcontroller-based IoT devices: the computational disparity among distributed learning nodes, commonly referred to as the “straggler” problem. Through rigorous experimentation with TDMiL, we conducted a comprehensive comparative analysis of multiple novel schemes devised to mitigate this issue. These experiments yield valuable insights into optimizing distributed learning processes within heterogeneous IoT ecosystems, paving the way for more efficient and robust AI deployments in resource-constrained environments.

To foster innovation and collaboration in this rapidly evolving domain, we have released our TDMiL implementation as open-source, portable software. This strategic decision empowers researchers and practitioners to not only reproduce our findings but also to extend and adapt our work to diverse hardware configurations and application scenarios. By leveraging either proprietary setups or established open-access IoT testbeds, the community can build upon our foundation, accelerating progress in the field of distributed AI for IoT systems.

Our work represents a significant stride towards harmonizing the ambitious demands of AI with the inherent limitations of IoT deployments. As these technological domains continue to evolve and converge, the framework we’ve developed serves as a robust platform for future innovations in distributed learning on resource-constrained devices. By addressing the unique challenges at the intersection of AI and IoT, we contribute to the broader vision of ubiquitous, intelligent systems that can operate efficiently at the edge of networks.

SOURCE CODE

Our code is accessible in this repository for testing and to encourage experimentation with various model architectures and datasets: <https://github.com/TinyPART/TDMiL-artifact>.

ACKNOWLEDGMENT

The article reflects only the authors’ views. MESRI and BMBF are not responsible for any use that may be made of the information it contains.

REFERENCES

- [1] L. Zeng, X. Chen, Z. Zhou, L. Yang, and J. Zhang, “CoEdge: Cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices,” *IEEE/ACM Trans. Netw.*, vol. 29, no. 2, pp. 595–608, Apr. 2021.
- [2] E. Samikwa, A. D. Maio, and T. Braun, “DISNET: Distributed micro-split deep learning in heterogeneous dynamic IoT,” *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6199–6216, Feb. 2024.
- [3] N. N. Alajlan and D. M. Ibrahim, “TinyML: Enabling of inference deep learning models on ultra-low-power IoT edge devices for AI applications,” *Micromachines*, vol. 13, no. 6, p. 851, May 2022.
- [4] J. Lin, W.-M. Chen, Y. Lin, C. Gan, and S. Han, “MCUNet: Tiny deep learning on IoT devices,” in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 11711–11722.
- [5] S. A. R. Zaidi, A. M. Hayajneh, M. Hafeez, and Q. Z. Ahmed, “Unlocking edge intelligence through tiny machine learning (TinyML),” *IEEE Access*, vol. 10, pp. 100867–100877, 2022.
- [6] V. Rajapakse, I. Karunanayake, and N. Ahmed, “Intelligence at the extreme edge: A survey on reformatable TinyML,” *ACM Comput. Surv.*, vol. 55, no. 13, pp. 1–30, Dec. 2023.
- [7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [8] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, “Federated learning: Strategies for improving communication efficiency,” 2016, *arXiv:1610.05492*.
- [9] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 429–450.
- [10] P. Pinyoanuntapong, P. Janakaraj, R. Balakrishnan, M. Lee, C. Chen, and P. Wang, “EdgeML: Towards network-accelerated federated learning over wireless edge,” *Comput. Netw.*, vol. 219, Dec. 2022, Art. no. 109396.
- [11] L. Giarretta and S. Girdzijauskas, “Gossip learning: Off the beaten path,” in *Proc. IEEE Int. Conf. Big Data*, Dec. 2019, pp. 1117–1124.
- [12] L. Wulfert, N. Asadi, W.-Y. Chung, C. Wiede, and A. Grabmaier, “Adaptive decentralized federated gossip learning for resource-constrained IoT devices,” in *Proc. 4th Int. Workshop Distrib. Mach. Learn.*, Dec. 2023, pp. 27–33.
- [13] Y.-G. Hong, C. Gomez, Y. Choi, A. R. Sangi, and S. Chakrabarti, “Applicability and use cases for IPv6 over networks of Resource-constrained Nodes (6lo),” RFC Editor, RFC 9453, Sep. 2023, doi: [10.17487/RFC9453](https://doi.org/10.17487/RFC9453). [Online]. Available: <https://www.rfc-editor.org/info/rfc9453>
- [14] L. Wulfert, C. Wiede, and A. Grabmaier, “TinyFL: On-device training, communication and aggregation on a microcontroller for federated learning,” in *Proc. 21st IEEE Interregional NEWCAS Conf.*, Jun. 2023, pp. 1–5.
- [15] X. Li, Z. Qu, B. Tang, and Z. Lu, “Stragglers are not disaster: A hybrid federated learning algorithm with delayed gradients,” 2021, *arXiv:2102.06329*.
- [16] *High Performance Open-Source Universal RPC Framework*, gRPC, 2018. [Online]. Available: <https://grpc.io/about/>
- [17] I. Fette and A. Melnikov, *The Websocket Protocol*, document RFC 6455, Dec. 2011. [Online]. Available: <http://www.rfc-editor.org/rfc/rfc6455.txt>
- [18] S. Liu, B. Guo, C. Fang, Z. Wang, S. Luo, Z. Zhou, and Z. Yu, “Enabling resource-efficient AIoT system with cross-level optimization: A survey,” *IEEE Commun. Surveys Tuts.*, vol. 26, no. 1, pp. 389–427, 1st Quart., 2024.
- [19] H. Hu, R. Peng, Y.-W. Tai, and C.-K. Tang, “Network trimming: A data-driven neuron pruning approach towards efficient deep architectures,” 2016, *arXiv:1607.03250*.
- [20] A. Kuzmin, M. van Baalen, M. Nagel, and A. Behboodi, “Quantized sparse weight decomposition for neural network compression,” 2022, *arXiv:2207.11048*.
- [21] M. Jaderberg, A. Vedaldi, and A. Zisserman, “Speeding up convolutional neural networks with low rank expansions,” 2014, *arXiv:1405.3866*.
- [22] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” 2021, *arXiv:2106.08295*.
- [23] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, “MCUNetV2: Memory-efficient patch-based inference for tiny deep learning,” in *Proc. Annu. Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021, pp. 2346–2358.
- [24] Y. Ding, L. Zhu, Z. Jia, G. Pekhimenko, and S. Han, “IoS: Inter-operator scheduler for CNN acceleration,” in *Proc. Mach. Learn. Syst.*, vol. 3, 2021, pp. 167–180.
- [25] Z. Huang, K. Zandberg, K. Schleiser, and E. Baccelli, “U-TOE: Universal TinyML on-board evaluation toolkit for low-power IoT,” in *Proc. 12th IFIP/IEEE Int. Conf. Perform. Eval. Model. Wired Wireless Netw. (PEMWN)*, Sep. 2023, pp. 1–6.
- [26] Z. Huang, K. Zandberg, K. Schleiser, and E. Baccelli, “RIOT-ML: Toolkit for over-the-air secure updates and performance evaluation of TinyML models,” in *Annals of Telecommunications*. Springer, 2024, pp. 1–15.

- [27] Y. D. Kwon, R. Li, S. I. Venieris, J. Chauhan, N. D. Lane, and C. Mascolo, "TinyTrain: Resource-aware task-adaptive sparse training of DNNs at the data-scarce edge," 2023, *arXiv:2307.09988*.
- [28] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, "On-device training under 256KB memory," 2022, *arXiv:2206.15472*.
- [29] H. Ren, D. Anicic, and T. A. Runkler, "TinyOL: TinyML with online-learning on microcontrollers," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.
- [30] S. G. Patil, P. Jain, P. Dutta, I. Stoica, and J. Gonzalez, "POET: Training neural networks on tiny devices with integrated rematerialization and paging," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, Jun. 2022, pp. 17573–17583.
- [31] M. M. Grau, R. P. Centelles, and F. Freitag, "On-device training of machine learning models on microcontrollers with a look at federated learning," in *Proc. ACM GoodIT*, 2021, pp. 198–203.
- [32] K. Kopparapu and E. Lin, "TinyFedTL: Federated transfer learning on tiny devices," 2021, *arXiv:2110.01107*.
- [33] H. Ren, D. Anicic, and T. A. Runkler, "TinyReptile: TinyML with federated meta-learning," 2023, *arXiv:2304.05201*.
- [34] H. Ren, X. Li, D. Anicic, and T. A. Runkler, "TinyMetaFed: Efficient federated meta-learning for TinyML," 2023, *arXiv:2307.06822*.
- [35] K. Zandberg, M. Gulati, G. Wunder, and E. Baccelli, "Model CBOR serialization for federated learning," 2024, *arXiv:2401.14056*.
- [36] E. Baccelli, C. Gündogan, O. Hahm, P. Kietzmann, M. S. Lenders, H. Petersen, K. Schleiser, T. C. Schmidt, and M. Wählisch, "RIOT: An open source operating system for low-end embedded devices in the IoT," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 4428–4440, Dec. 2018.
- [37] C. Bormann, M. Ersue, and A. Keranen, *Terminology for Constrained-Node Networks*, document RFC 7228, 2014.
- [38] *TensorFlow Federated*. Accessed: Apr. 25, 2024. [Online]. Available: <https://github.com/tensorflow/federated>
- [39] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, J. Fernandez-Marques, Y. Gao, L. Sani, K. Hei Li, T. Parcollet, P. Porto Buarque de Gusmão, and N. D. Lane, "Flower: A friendly federated learning research framework," 2020, *arXiv:2007.14390*.
- [40] *Protocol Buffers*. Accessed: Apr. 25, 2024. [Online]. Available: <https://protobuf.dev/>
- [41] J. Hui, D. Culler, and S. Chakrabarti, "6LoWPAN: Incorporating IEEE 802.15.4 into the IP architecture internet protocol for smart objects (IPSO) alliance," Univ. California, Berkeley, CA, USA, White Paper 3, 2009.
- [42] F. Palmese, E. Longo, A. E. C. Redondi, and M. Cesana, "CoAP vs. MQTT-SN: Comparison and performance evaluation in publish-subscribe environments," in *Proc. IEEE 7th World Forum Internet Things (WF-IoT)*, Jun. 2021, pp. 153–158.
- [43] Z. Shelby, K. Hartke, and C. Bormann, *The Constrained Application Protocol (CoAP)*, document RFC 7252, 2014.
- [44] A. Faridi, M. R. Palattella, A. Lozano, M. Dohler, G. Boggia, L. A. Grieco, and P. Camarda, "Comprehensive evaluation of the IEEE 802.15.4 MAC layer performance with retransmissions," *IEEE Trans. Veh. Technol.*, vol. 59, no. 8, pp. 3917–3932, Oct. 2010.
- [45] K. Zandberg, K. Schleiser, F. Acosta, H. Tschofenig, and E. Baccelli, "Secure firmware updates for constrained IoT devices using open standards: A reality check," *IEEE Access*, vol. 7, pp. 71907–71920, 2019.
- [46] P. van der Stok, C. Bormann, and A. Sehgal, *Patch and Fetch Methods for the Constrained Application Protocol (CoAP)*, document RFC 8132, 2017.
- [47] C. Amsüss, Z. Shelby, M. Koster, C. Bormann, and P. V. der Stok, *Constrained RESTful Environments (CoRE) Resource Directory*, document RFC 9176, 2022.
- [48] K. Hartke, *Observing Resources in the Constrained Application Protocol (CoAP)*, document RFC 7641, Sep. 2015.
- [49] S. Ramírez, *FastAPI*. Accessed: Apr. 25, 2024. [Online]. Available: <https://github.com/tiangolo/fastapi>
- [50] C. Amsüss and M. Wasilak, "AioCoAP: Python CoAP library," *Energy Harvesting Solutions*, vol. 1570, 2013.
- [51] R. A. Fisher, "Iris," UCI Mach. Learn. Repository, 1936, doi: [10.24432/C56C76](https://doi.org/10.24432/C56C76).
- [52] S. Aeberhard and M. Forina, "Wine," UCI Mach. Learn. Repository, 1992, doi: [10.24432/C5PC71](https://doi.org/10.24432/C5PC71).
- [53] M. Zwitter and M. Soklic, "Breast cancer," UCI Mach. Learn. Repository, 1988, doi: [10.24432/C51P4M](https://doi.org/10.24432/C51P4M).
- [54] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," *J. Mach. Learn. Res.*, vol. 3, pp. 993–1022, Jan. 2003.
- [55] Y. LeCun, C. Cortes, and C. Burges. (2010). *Mnist Handwritten Digit Database*. ATT Labs. [Online]. Available: <http://yann.lecun.com/exdb/mnist>
- [56] *RIOT Native: Running a RIOT Instance in a Linux Process*. Accessed: Apr. 25, 2024. [Online]. Available: https://doc.riot-os.org/group_boards__native.html
- [57] *IoT-Lab Large Scale Open-Access Testbed*. Accessed: Apr. 25, 2024. [Online]. Available: <https://www.iot-lab.info/>
- [58] C. Adjih, E. Baccelli, E. Fleury, G. Harter, N. Mitton, T. Noel, R. Pissard-Gibollet, F. Saint-Marcel, G. Schreiner, J. Vandaele, and T. Watteyne, "FIT IoT-LAB: A large scale open experimental IoT testbed," in *Proc. IEEE WF-IoT*, Dec. 2015, pp. 459–464.
- [59] G. Guennebaud. (2010). *Eigen V3*. [Online]. Available: <http://eigen.tuxfamily.org>
- [60] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [61] A. Howard, M. Sandler, B. Chen, W. Wang, L.-C. Chen, M. Tan, G. Chu, V. Vasudevan, Y. Zhu, R. Pang, H. Adam, and Q. Le, "Searching for MobileNetV3," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 1314–1324.
- [62] L. Prechelt, "Early stopping-but when?" in *Neural Networks: Tricks of the Trade*. Berlin, Germany: Springer, 2002, pp. 55–69.



MAYANK GULATI received the double master's degree in intelligent autonomous systems from KTH Royal Institute of Technology, Stockholm, Sweden, and Technische Universität Berlin, Germany, in 2021, and the Engineering degree in mechatronics, robotics, and automation engineering from the University of Petroleum and Energy Studies, Dehradun, India, in 2017. He has been a Research Associate at Freie Universität Berlin, Berlin, Germany, since October 2021. Prior to this,

he was a Research Assistant at Fraunhofer Heinrich Hertz Institute (HHI), Berlin, Germany, from January 2021 to October 2021. He completed a research internship at Ericsson, Stockholm, Sweden, from February 2020 to October 2020, where he worked on reinforcement learning approaches in cellular networks. His research interests include machine learning topics, such as federated learning, edge computation, privacy, and their applications.



KOEN ZANDBERG received the M.Sc. degree in electrical engineering from the University of Twente. He is currently pursuing the Ph.D. degree in embedded systems with Freie Universität Berlin, with a focus on secure IoT software and low-power wireless networking. From 2021 to 2023, he was with Inria, France, as a Research Engineer. He is also a maintainer of the RIOT open-source project and actively contributes to over-the-air updates and the networking subsystem.



ZHAOLAN HUANG received the M.Sc. degree in electrical engineering and information technology (automation) from Technische Universität Darmstadt, Germany, in 2021. He is currently pursuing the Ph.D. degree with Freie Universität Berlin, Germany. He is also a Research Assistant with Freie Universität Berlin. His research interests include TinyML systems and resource optimization of machine learning models for low-power IoT devices. He primarily focuses on exploring

the capabilities of on-device inference and training of machine learning models and aims at integrating machine learning support into RIOT operating systems.



GERHARD WUNDER (Senior Member, IEEE) received the degree (Hons.) in electrical engineering and the Ph.D. degree (summa cum laude) from TU Berlin, in 1999 and 2003, respectively. He is a Distinguished Cybersecurity and AI Professor at Freie Universität Berlin and a Heisenberg Fellow of Deutsche Forschungsgemeinschaft (DFG). He held roles at the Fraunhofer Heinrich-Hertz-Institut and became a Privatdozent at TU Berlin, in 2007. He has visited Georgia Institute of Technology and Stanford University, consulted at Bell Labs, and in 2014, became the first communication engineer to receive the Heisenberg Fellowship. He has received research fellowships from DFG, led projects, such as FP7 5GNOW and PROPHYLAXE on IoT Physical Layer Security, and chaired workshops and conference tracks on ML/AI and 5G/6G technologies. He has been nominated for awards, such as the Deutscher Zukunftspreis, in 2017.



EMMANUEL BACCELLI received the Ph.D. degree from École Polytechnique, Paris, France, in 2006, and the Habilitation degree from Université Pierre et Marie Curie, Paris, France, in 2012. After working at AT&T Labs, Florham Park, NJ, USA, and Metro Optix Inc., Silicon Valley, as a Software Engineer, until 2002, he joined Hitachi Europe as a Research Engineer. He is currently a Researcher at Inria, France, and a Professor at Freie Universität Berlin. He is also the co-founder and coordinator of the open-source community developing the RIOT operating systems. He has authored or co-authored dozens of research papers in the field's relevant conferences and journals, and contributes continuously to standardization efforts within the Internet Engineering Task Force (IETF), co-authoring several RFCs.

...



CEDRIC ADJIH received the degree from ECP (now CentraleSupélec, part of Université Paris-Saclay, France), in 1994, and the Ph.D. degree from the University of Versailles Saint-Quentin-en-Yvelines, France, in 2001. He is currently a Research Scientist at Inria, Research Center of Saclay-Île-de-France. His research interests include random access technology in the IoT; and in general, communications, the IoT, and machine learning for networking and embedded systems.